Reflection: what we would improve with more time

Group 4 · Swagata Bhowmik (B053) and Sumit Kothari (B062) · Project 4 Capstone

What worked

The most useful result is not the accuracy figure (validation MAE 5.74, final test 5.78 lakhs) but that every release decision is automatic and explainable. The gates rejected a bad data file and a weak model while the live endpoint kept serving, a CI pull request with an under-fitting model ($\alpha = 1000$) was blocked before it could reach main, and every deployed model can be traced to its commit, data hash and MLflow run. The dashboard made these behaviours visible to non-engineers.

What the incoming batches taught us

Batch 2 drifted (smaller areas) without a performance alert (6.87 lakhs), while batch 3 drifted in four inputs and its error doubled to 13.11. Input drift and model failure are related but not the same, so we report them separately. Our thresholds were chosen after looking at the data, so these batches demonstrate the policy rather than validate it.

What we would improve

1. Serving capacity. Under 5 concurrent clients the free host reached p95 857 ms end to end against a 200 ms target. We would move to a host with more CPU and several workers, add autoscaling, and repeat the load test with longer, mixed and cold-start workloads.

2. Persistent production state. Render Free rebuilds the model on every deploy, so the incumbent comparison only works across releases in the local/Compose setup. We would keep the MLflow registry and model artifacts in a shared store so every cloud release is compared with the live model, and host MLflow and Grafana with authentication.

3. Safer promotion. Replace the file pointer plus registry alias with a single promotion record and crash recovery, and add blue/green or canary releases with automatic rollback on failed health checks instead of a manual rollback workflow.

4. Better monitoring. Calibrate drift thresholds on many reference samples to estimate false-alarm rates, add per-zone error and minimum sample sizes, and send alerts to a notification channel with an acknowledged incident process.

5. Data and orchestration. Use dated listings for time-based evaluation, version data with DVC once files grow, and move to Prefect or ZenML when scheduling, retries and multiple workers are needed.

How we worked

We used an AI coding assistant for much of the implementation. Our own responsibility was to define the requirements and thresholds, check every claim against recorded evidence, and be able to explain, run and modify each stage ourselves; we deliberately kept the orchestration small so that we can. Each of us should still rehearse a live change (for example, editing a threshold in config.yaml and showing the gate react) before the demonstration.